CS637 Project

SEQUIN: Sequential N-k Attacks in Electric Power Grids



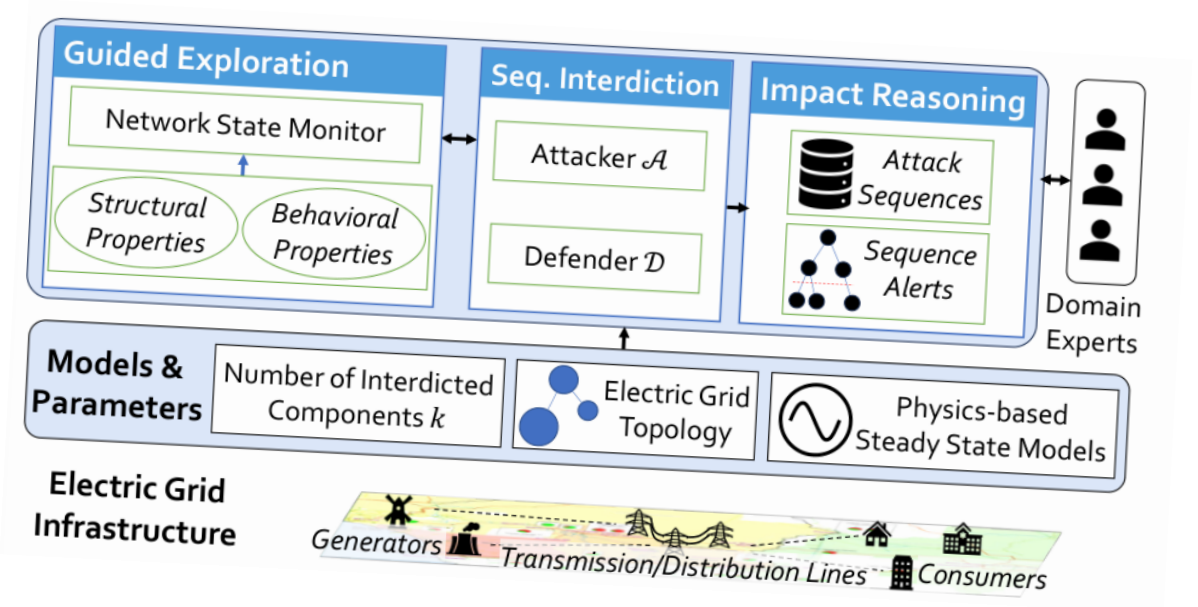
A
B
C
D
E
F

SEQUIN

A Network Science and Physics-based Approach to Identify Sequential N-k Attacks in Electric Power Grids

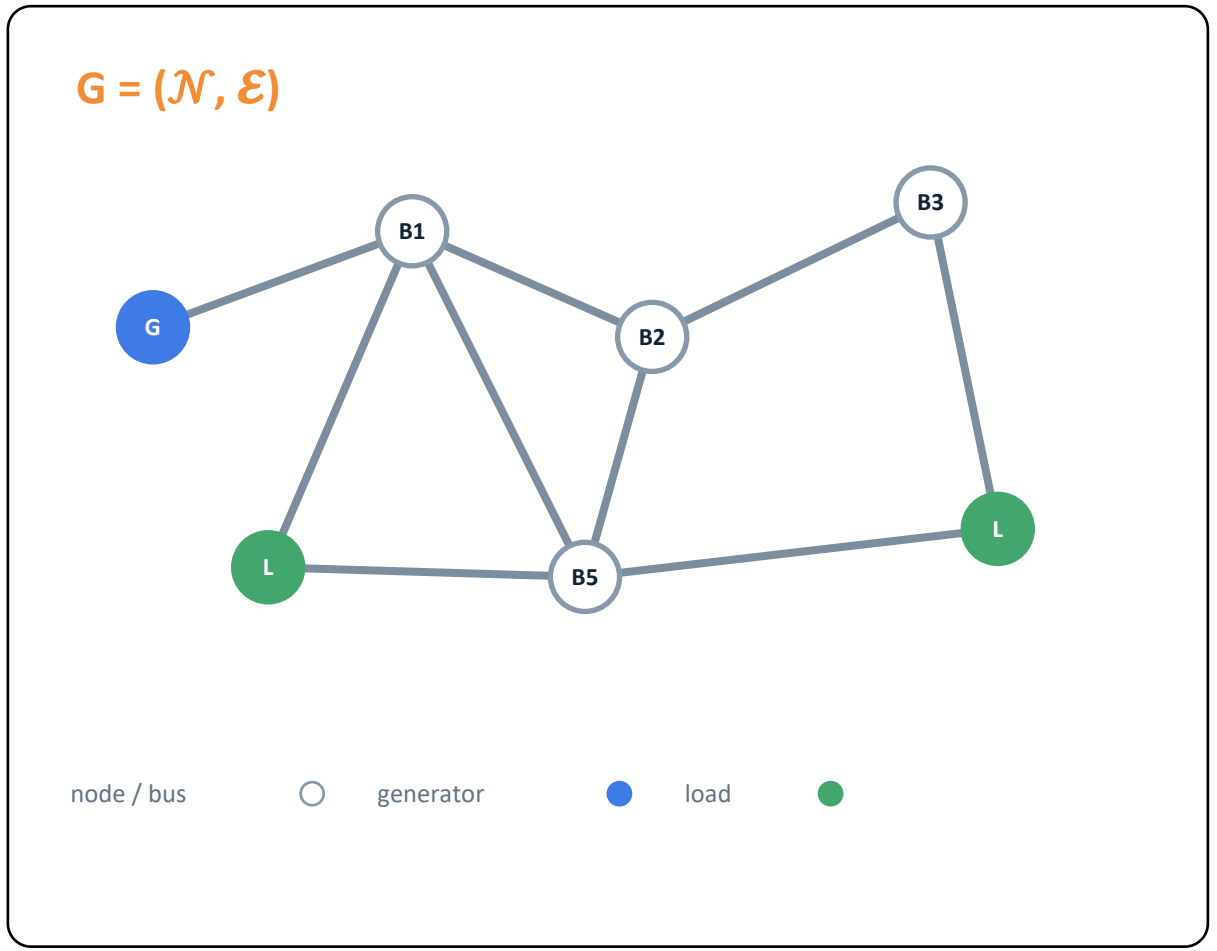
ICCPS 2025

- Andrew Chio • Russell Bent
- Kaarthik Sundar • Nalini Venkatasubramanian



CONTEXT

Modelling Power Grid as Network



Paper Symbols

θ_i	voltage phase angle
p_i^g	generator output
p_i^d	load demand
p_i^s	load shed
p_{ij}	actual line power flow
b_{ij}	inverse reactance / susceptance
t_{ij}	thermal rating / capacity
δ_i	generator ramp bound

CONTEXT

Physics Constraints

01 Bus balance

$$p^g_i - p^d_i + p^s_i = \Sigma\, p_{ij} - \Sigma\, p_{ji}$$

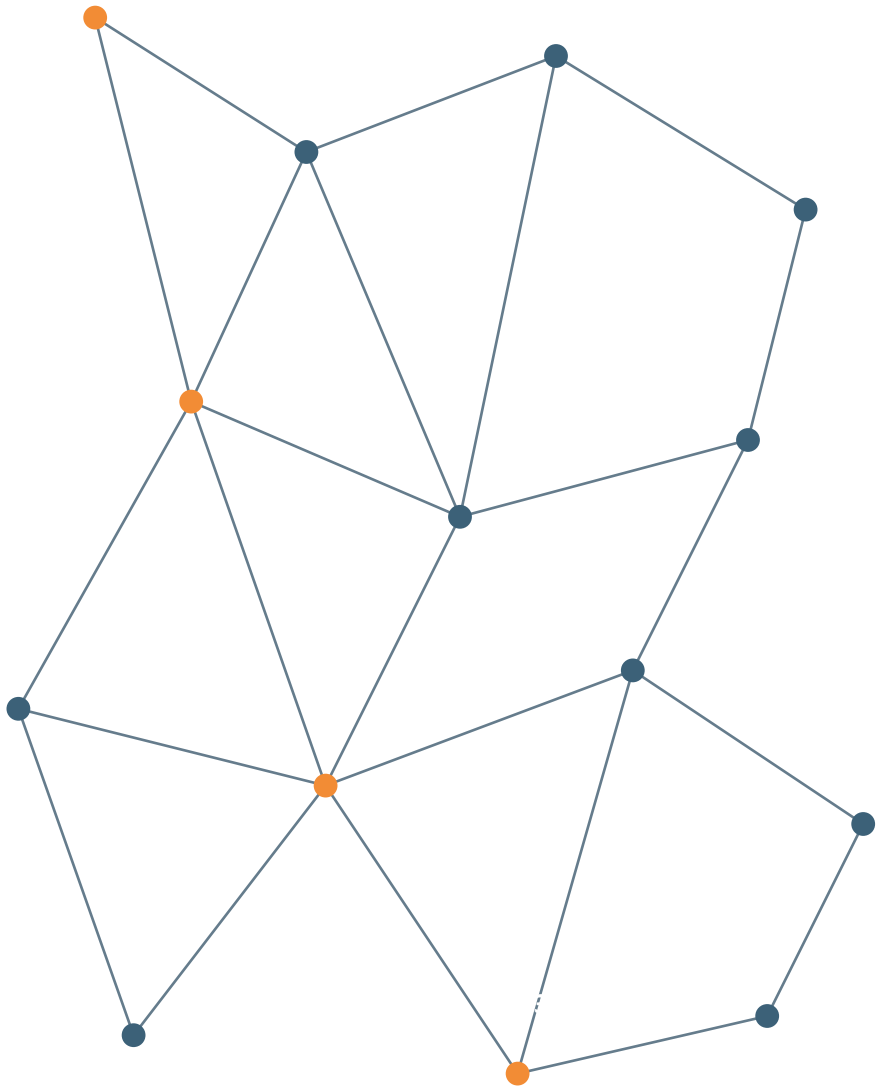
Power cannot appear or disappear at a bus.

02 DC line flow

$$p_{ij} = b_{ij}(\theta_i - \theta_j)$$

$$|p_{ij}| \leq t_{ij}$$

Capacity t_{ij} bounds the flow p_{ij}.



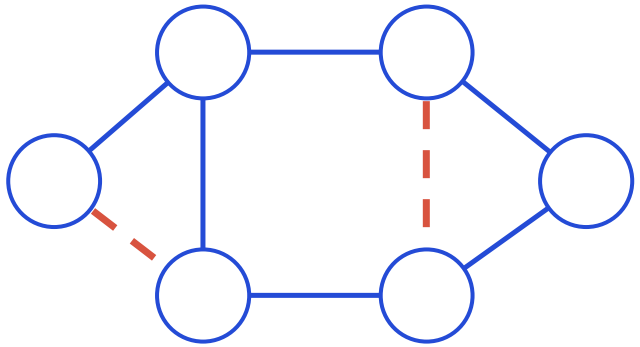
MOTIVATION FOR SEQUIN

Failures unfold while the grid is responding.

Hurricane Harvey, 2017
Moore County attack, 2022

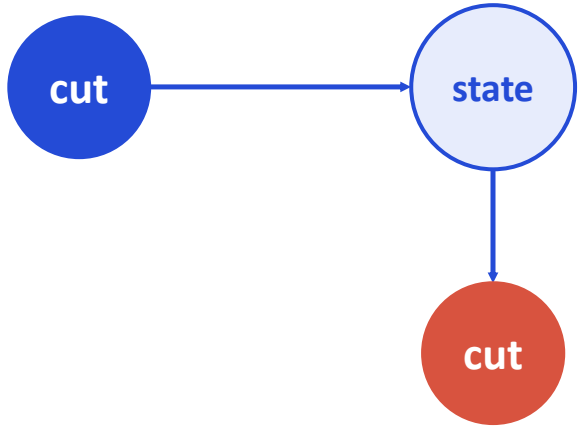
Traditional vs Sequential

Simultaneous N-k



Fail a set of k components
Then solve one response

Sequential N-k



Fail a component
Respond before the next failure

A + B + C ≠ A → B → C

CPS intuition: the plant is changing while the adversary is still acting.

MOTIVATION FOR SEQUIN

Why Not Try Every Sequence?

Because

$P(m, k) = m! / (m - k)!$

Case39: 46 lines, k = 3

91,080.

Case240 : 448 lines, k = 8

38.2 Trillion

SEQUIN

SEQUIN = Search + Physics + Interpretation

Guided Exploration

Which failures are worth exploring?

Attacker

Sequential interdiction

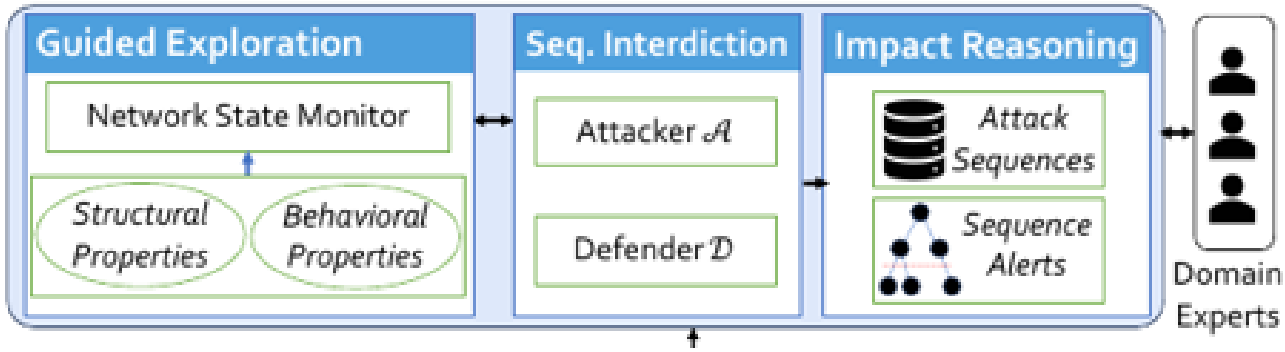
What physically happens after each failure?

Defender

Impact reasoning

What dangerous patterns appear across sequences? Build a Attack tree

Analysis by Domain Experts



SEQUIN Core Algorithm

ATTACKER $\mathcal{A}$

maximize final disruption

DEFENDER $\mathcal{D}$

maximize load served

01 Solve healthy steady state

02 Compute PF / criticality / TW–LD

03 Take top expl starting candidates

04 Attack one candidate

05 Defender solves new state

06 Update candidate set

07 Cut Impact picks next attack

08 Repeat until k failures

Algorithm 1 Guided Exploration

Input: G=(N,E), k, expl

Output: attack sequences S

```
σ ← SolveSteadyState(G)
props ← flow, criticality, TW/LD
starts ← top expl values from props
S ← ∅
for x₀ in starts:
    x ← [x₀]
    for r = 2 ... k:
        σ ← PlayInterdictionModel(G, x, σ)
        nextcomps ← UpdateTopValues(σ, props)
        x' ← argmax CutImpact(nextcomps, σ)
        append x' to x
    add x to S
return S
```

Not one greedy path: several promising first lines create a set S of sequences.

SEQUIN

How Does SEQUIN Shortlist Attacks?

1	2	3	4	5
Power Flow	Criticality	Transmission Width	Load Density	Cut Impact
Is this line carrying a lot of power right now?	How little operating margin remains?	How easily can a node be isolated?	How easily can a high-load region be isolated?	If this candidate is cut now, how much load is shed after response?

cheap metrics → shortlist → Cut Impact → next line

SEQUIN

How Did the Authors Test SEQUIN?

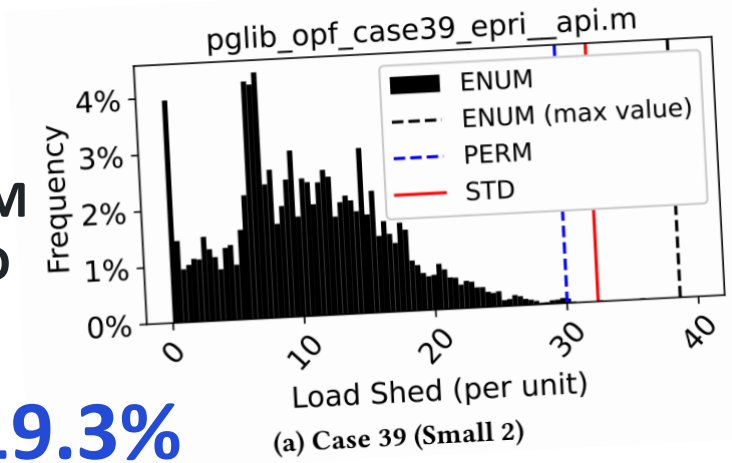
Benchmark networks

Case 14	Case 39	Case 60	Case 118	Case 162	Case 240
20 branches	46 branches	88 branches	186 branches	284 branches	448 branches
5 generators	10 generators	23 generators	54 generators	12 generators	143 generators

Compared against

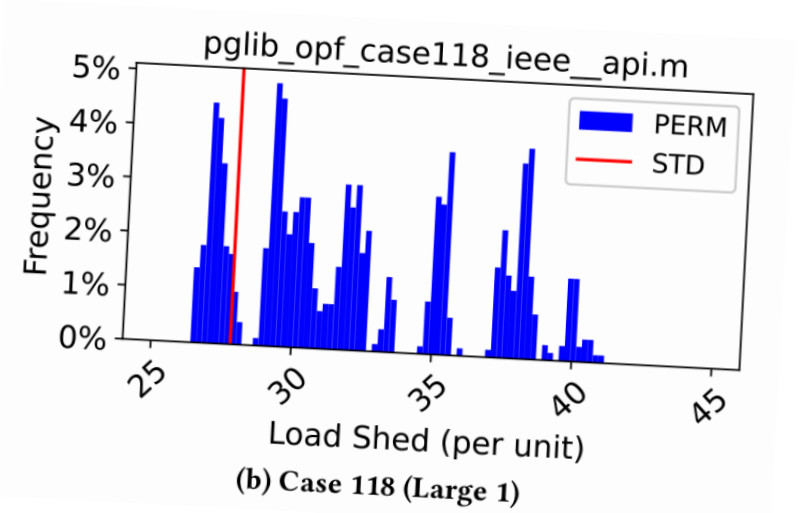
STD	PERM	ENUM	FLOW	SHED	CRIT	SEQUIN
simultaneous	reorder STD	all orderings	highest flow	largest impact	criticality	guided

Case39: best ENUM above STD



(a) Case 39 (Small 2)

+19.3%



(b) Case 118 (Large 1)

Case118: best PERM above STD

+47%

Reproduction

Results from executing Authors' Artifacts

sequin_rep — bash

```
$ cd /iccps-sequin
$ julia --project=. src/REPmain.jl \
    --use_separate_budgets --rerun \
    --case pglib_opf_case14_ieee.m \
    --problem SEQUIN -l 2 -k 2

[ Gurobi ]  Optimizing model ...
[ status ]  OPTIMAL
[ attack ]  [[7], [1]]
[ load shed ]  2.531

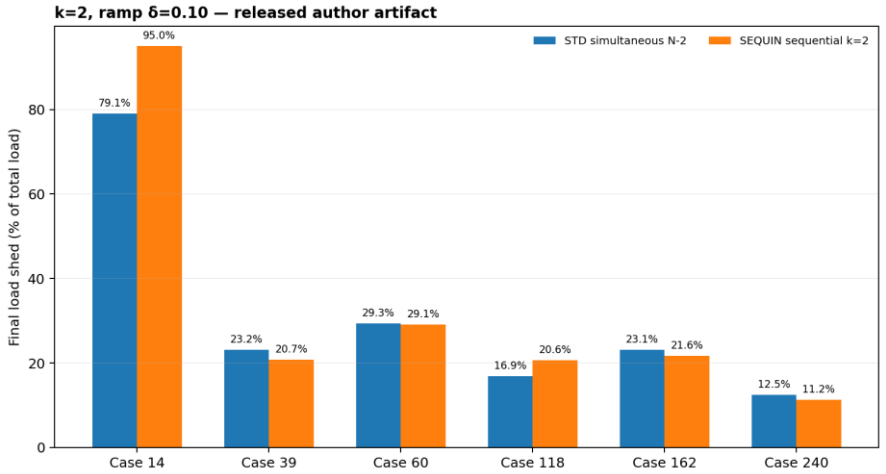
✓ Julia environment
✓ PowerModels / JuMP
✓ licensed Gurobi
✓ fresh SEQUIN path
```

authors' official reproduction pipeline

```
$ python src/experiments/
  REP_make_plots.py cache-authors

Case 14    ✓  exp1 / ramp plots
Case 39    ✓  exp0 / exp1 / ramp
Case 60    ✓  exp1 / ramp plots
Case 118   ✓  exp0 / exp1 / ramp
Case 162   ✓  exp1 / ramp plots
Case 240   ✓  exp1 / ramp plots

✓ 6 / 6 benchmark networks
✓ official figures regenerated
```



Implementation

Scratch Python implementation — model, search, and solver

What we built from scratch

grid_model.py

MATPOWER parser • DC balance/flow • thermal + angle limits
load shedding • islands • ramping • generator on/off

search_methods.py

exact simultaneous / sequential • FLOW • IMPACT • PERM
SEQUIN-style top-flow shortlist + cut-impact lookahead

network_metrics.py

criticality • branch margin • load density
transmission-width heuristic

Defender: maximize served load subject to DC physics + recourse

$p_{ij} = b_{ij}(\theta_i - \theta_j)$ • $|p_{ij}| \le t_{ij}$ • ramp + on/off

Core guided-search loop

```
for line in candidates:
    cand = defend(case, failed + [line],
                  state, ramp)
    impact[line] = shed_pu(case, cand)

next_line = max(impact, key=impact.get)
failed.append(next_line)
state = defend(case, failed, state, ramp)
```

Input

6 PGLib / MATPOWER cases

Solver

SciPy MILP / HiGHS

Experiments

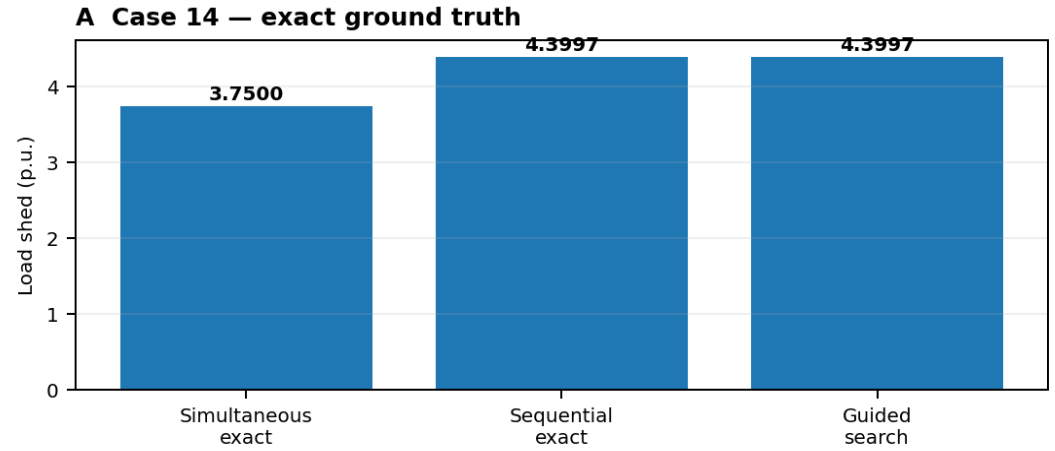
k • ramp • sequence • perturbation

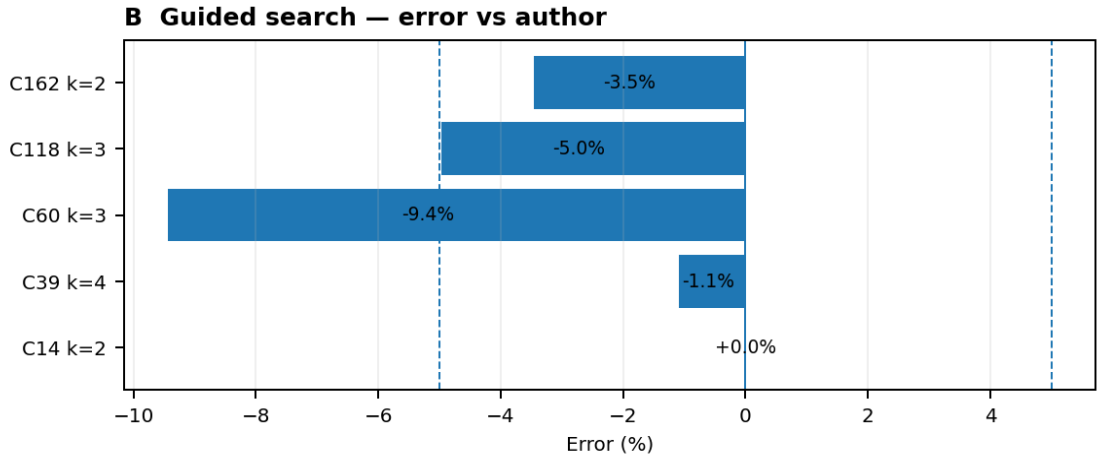
Independent implementation: no PowerModels/JuMP/Gurobi dependency.

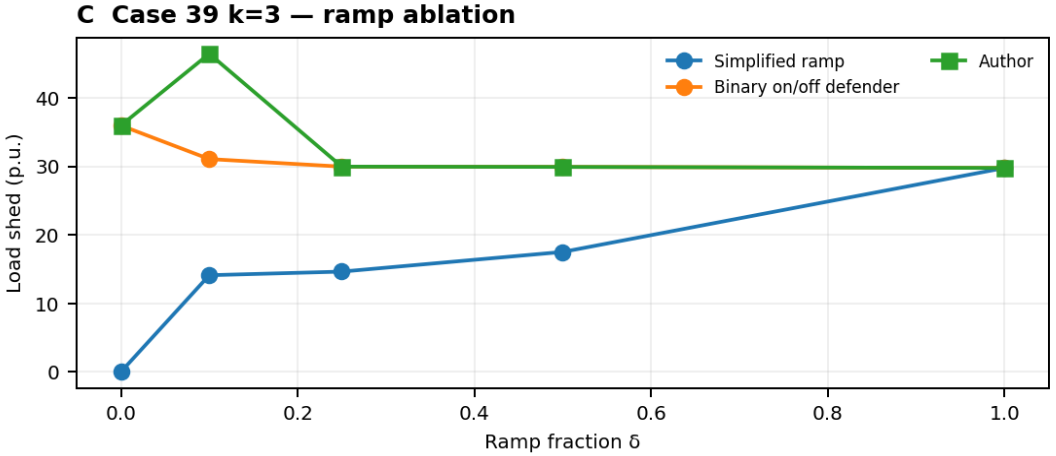
Results

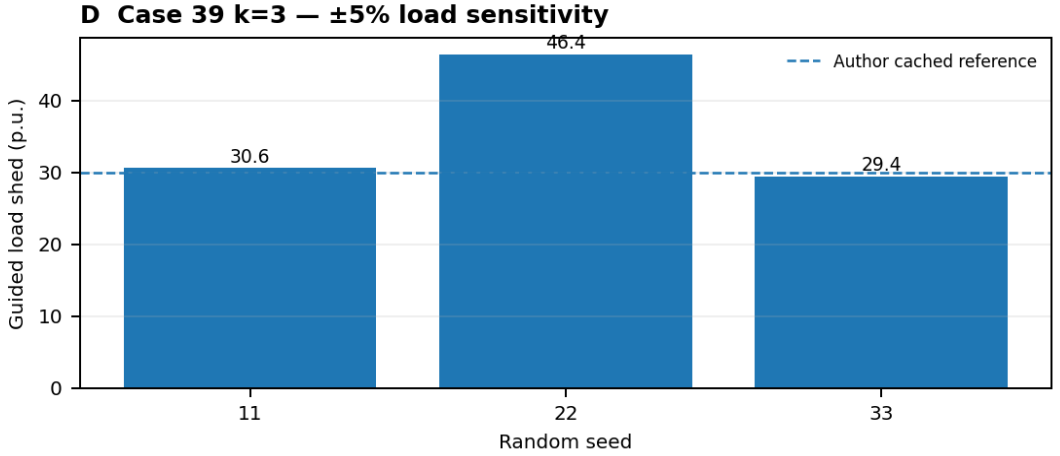
Independent Python results — verification, sensitivity, and failure modes

Four checks answer four different questions







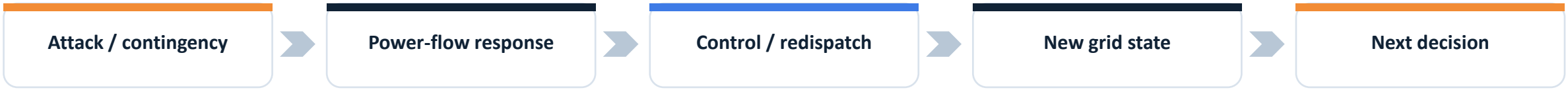


- Exact optimum recovered on Case14 • Case39 guided ≈ 0–1.1% error • binary generator recourse matters • ±5% loads can swing damage from 29.4 → 46.4 p.u.

Impact

From sequential-attack analysis to practical grid-resilience use cases

CPS problem: every decision changes the physical state seen by the next decision



Grid operator / planner

- screen dangerous ordered N-k contingencies
- rank corridors for hardening / redundancy
- test emergency redispatch + load-shed limits
- compare short vs long recovery intervals

Security / preparedness

- identify critical attack prefixes, not only line sets
- prioritize protection of high-consequence assets
- build red-team stress tests and operator drills
- explain why the same failures in another order differ

CPS relevance **Discrete adversarial events + continuous network physics + feedback control form a closed loop. Sequence, timing, and controller limits jointly determine resilience.**

CRITIQUE

Limitations and three extensions

01

Line protection

No investment selection

Which few lines should we protect?

02

Response timing

Fixed response epochs

What changes when failures arrive closer together?

03

Critical services

All megawatts valued equally

Which services should retain power first?

Three implemented studies on the group repository’s SciPy defender

EXTENSION 1

Scenario-aware line protection

The attacker adapts after protection

01 **Build complete attack tables**

Every eligible ordered pair, for each design demand.

02 **Choose the next protected line**

Minimize the worst loss across design scenarios.

03 **Search again after protection**

Re-evaluate every pair that remains eligible.

Protection budget

q = 0–3

Equal cost per protected line

Protected lines become ineligible for the modeled failure.

Perfect hardening is a modeling assumption. Greedy selection has no general global guarantee.

EVALUATION

Protection evaluation protocol

Case 14

380

ordered failure pairs

Case 39

2,070

ordered failure pairs

Design / test split

Design seeds 0, 1, 2; test seeds 100–104. Demand varies by ±5%.

Six baseline families

Random ×20, flow, betweenness, single-scenario, simultaneous design and exact outer design ($q \leq 2$).

Independent replay

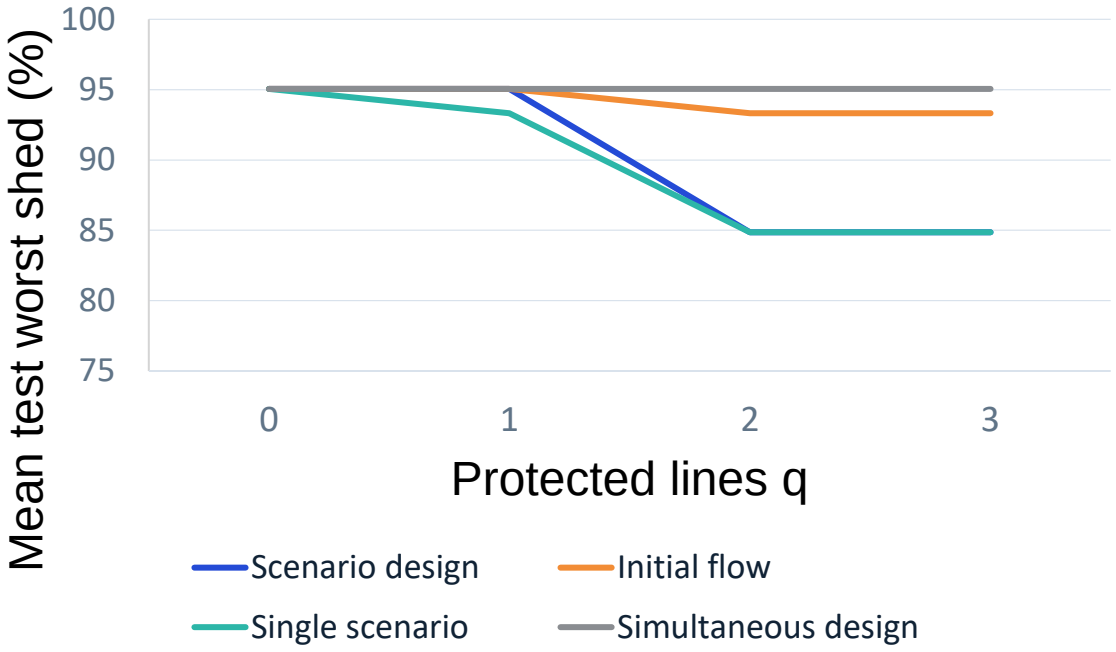
Fresh exhaustive post-protection searches. Test outcomes never choose the plan.

Attack order matters: (A, B) and (B, A) can produce different physical responses

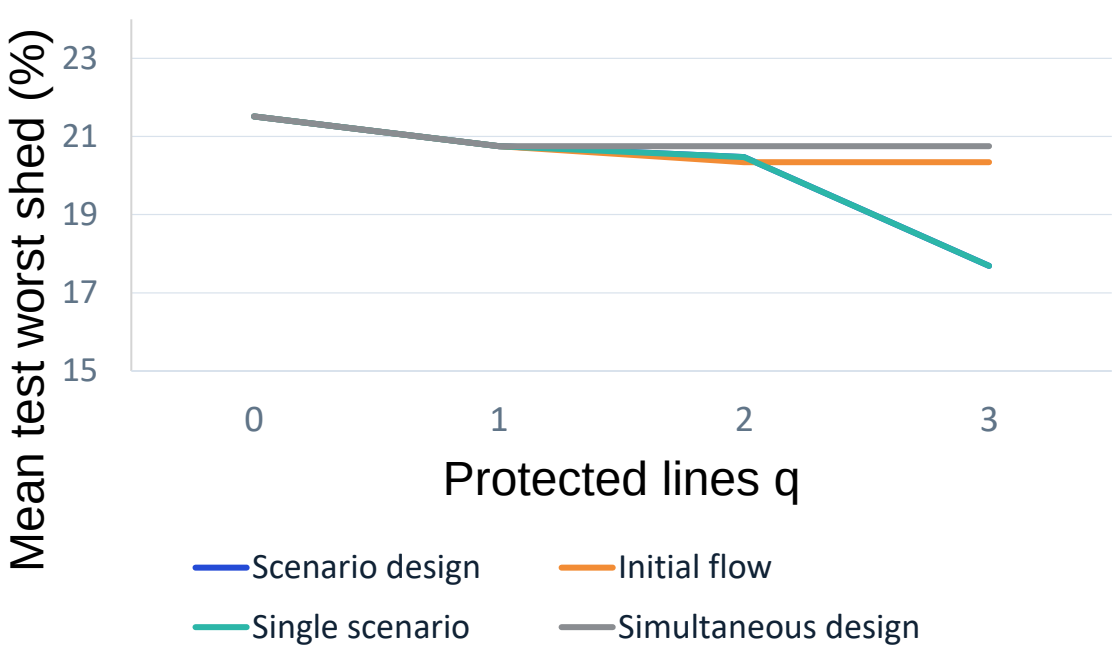
RESULTS

Protection on unseen demands

Case 14



Case 39



84.87% vs 93.32%

Scenario design vs flow, q = 2

20.47% vs 20.35%

Flow is slightly better at q = 2

Lower loss is better. Points average the worst eligible attack in each held-out demand scenario.

ROBUSTNESS

Protection under shifted conditions



10 / 36

Feasible comparisons where scenario design loses to flow

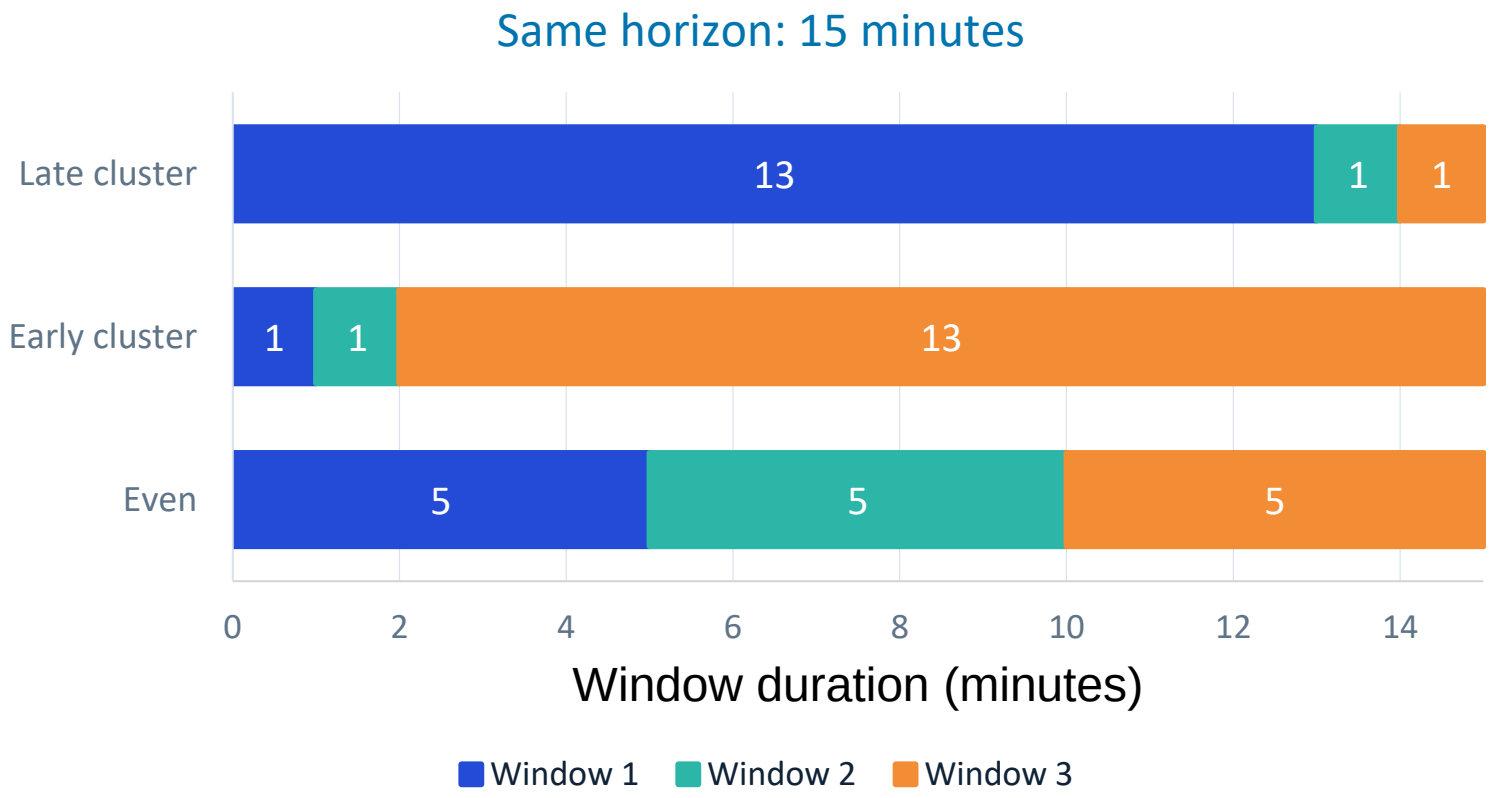
35.48 pp

Largest excess loss against flow

Plans stay fixed. 36 policy rows have infeasible intact starts and remain separate from attack losses.

EXTENSION 2

Variable response windows



Three decisions each

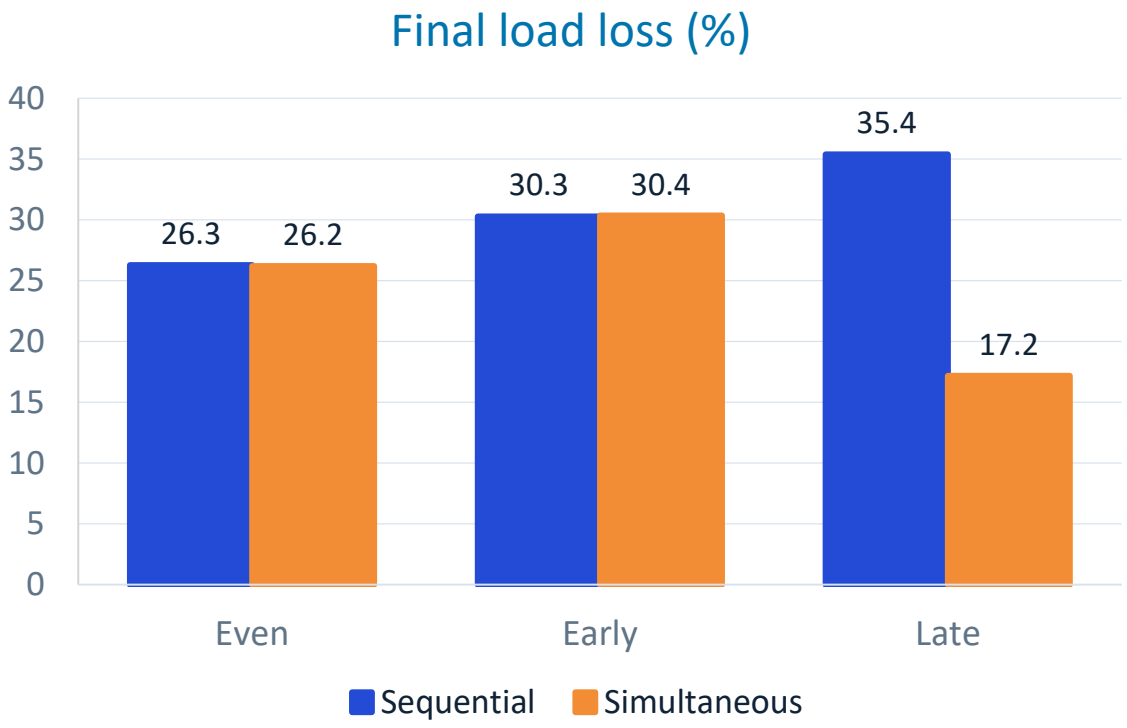
Sequential: one new failure in each window.

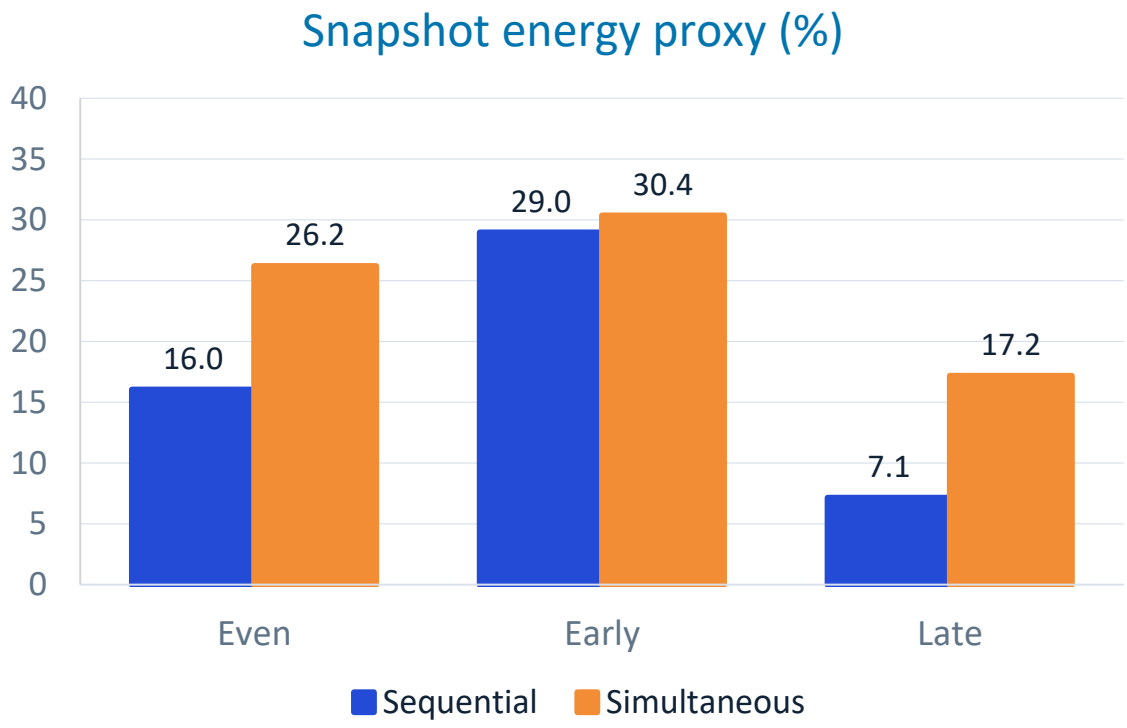
Simultaneous: all failures in the first window, then two more responses.

Longer windows allow more generator adjustment. Sensitivity horizons: 7.5, 15 and 30 minutes.

TIMING

Final loss and the energy proxy





26.3% to 35.4%

Sequential final loss: even vs late

16.0% to 7.1%

Sequential snapshot proxy: even vs late

Case 118, H = 15 min. Mean per-seed cohort maxima. The proxy does not integrate continuous ramps.

EXTENSION 3

Critical-service operating objectives

Total-tight

Every MW counts equally

Matched 10^{-8} solver precision

Weight 2

Critical MW counts twice

A moderate service preference

Weight 10

Critical MW counts ten times

A stronger service preference

Critical-first

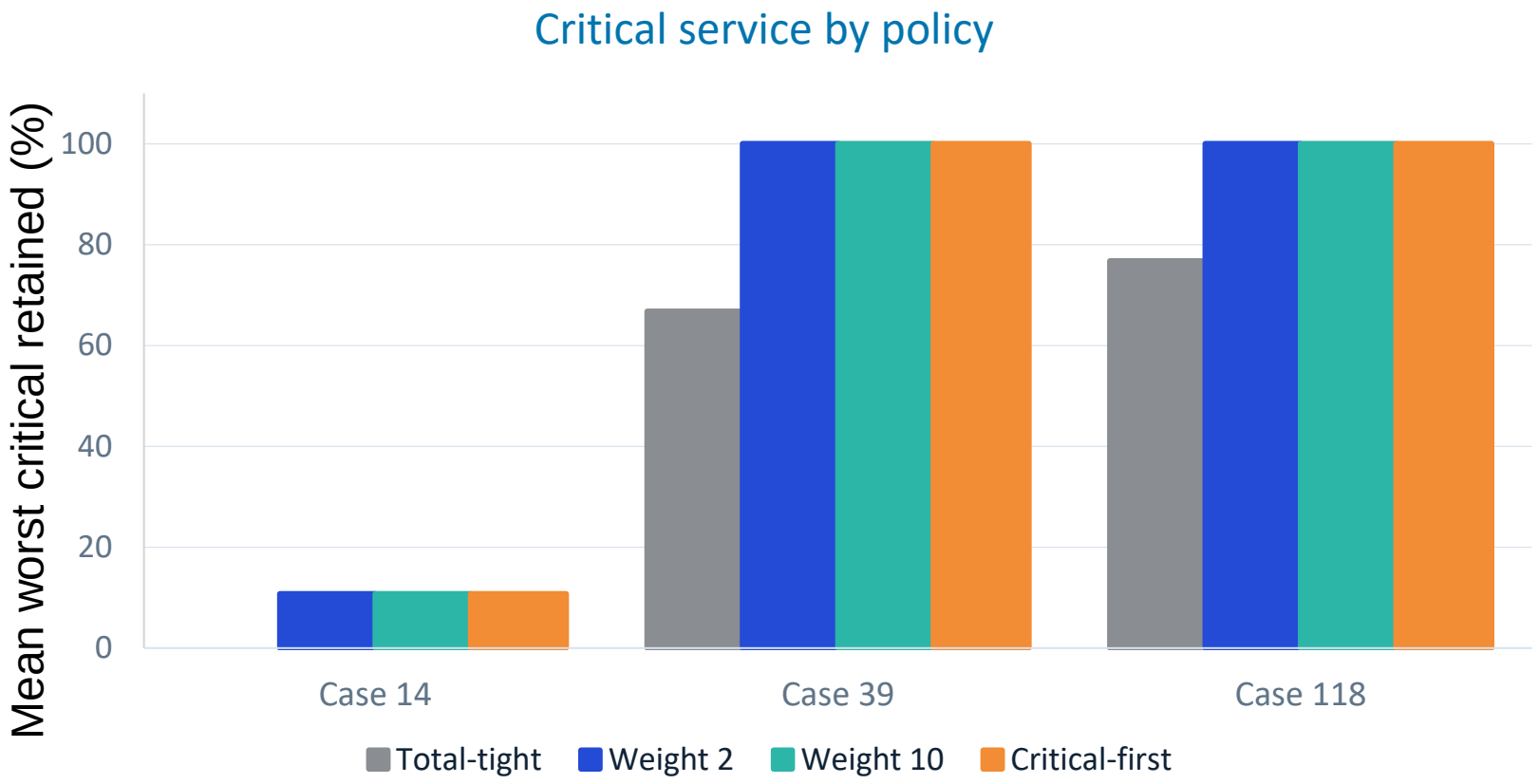
Maximize critical service first

Then maximize total service

Synthetic groups of positive-demand buses. All policies share initial dispatch and attack sequences.

PRIORITY

Critical service at the nominal setting



67% to 100%

Case 39 critical service retained

77% to 100%

Case 118 critical service retained

25% of positive-demand buses, ramp 0.1. Larger cases use a shared sampled cohort, not all attacks.

TRADE-OFFS

Priority gains and adverse cases

Selected positive case

Case 39, attack [39, 14]

+1,573 MW

more critical demand preserved

+121 MW total loss

Selected adverse case

Case 14, attack [4, 2]

−240 MW

critical service lost versus baseline

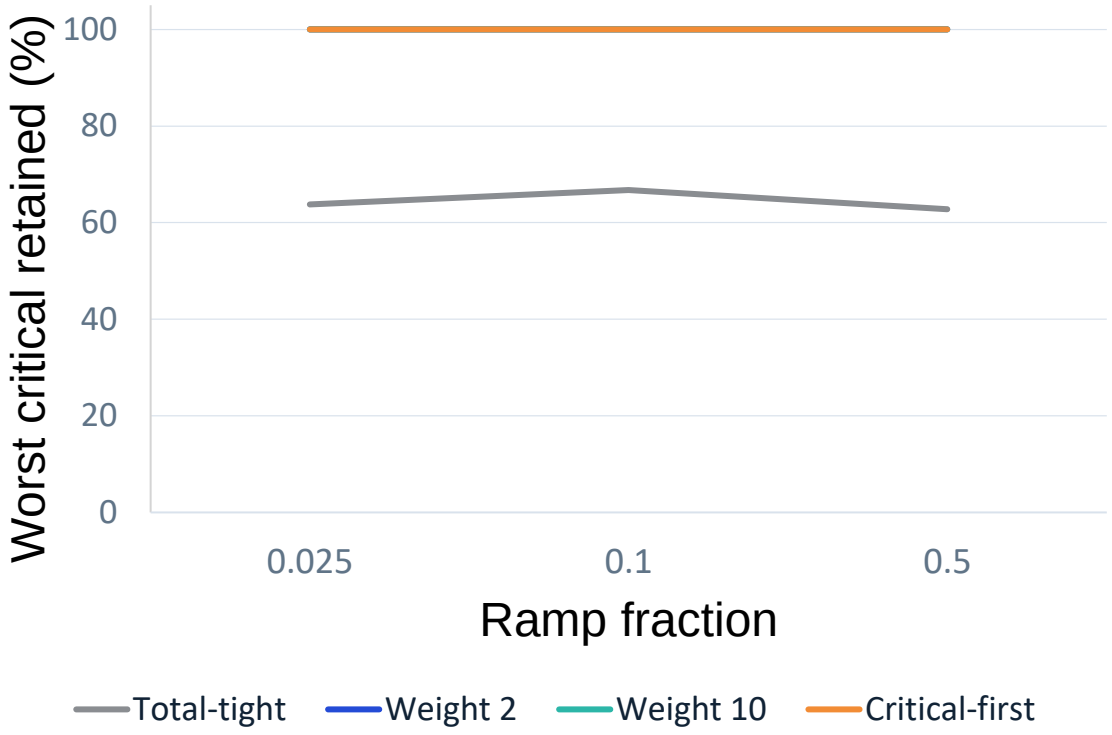
+323 MW total loss

Selected extremes against total-tight. Current-step priorities can create a worse state after a later failure.

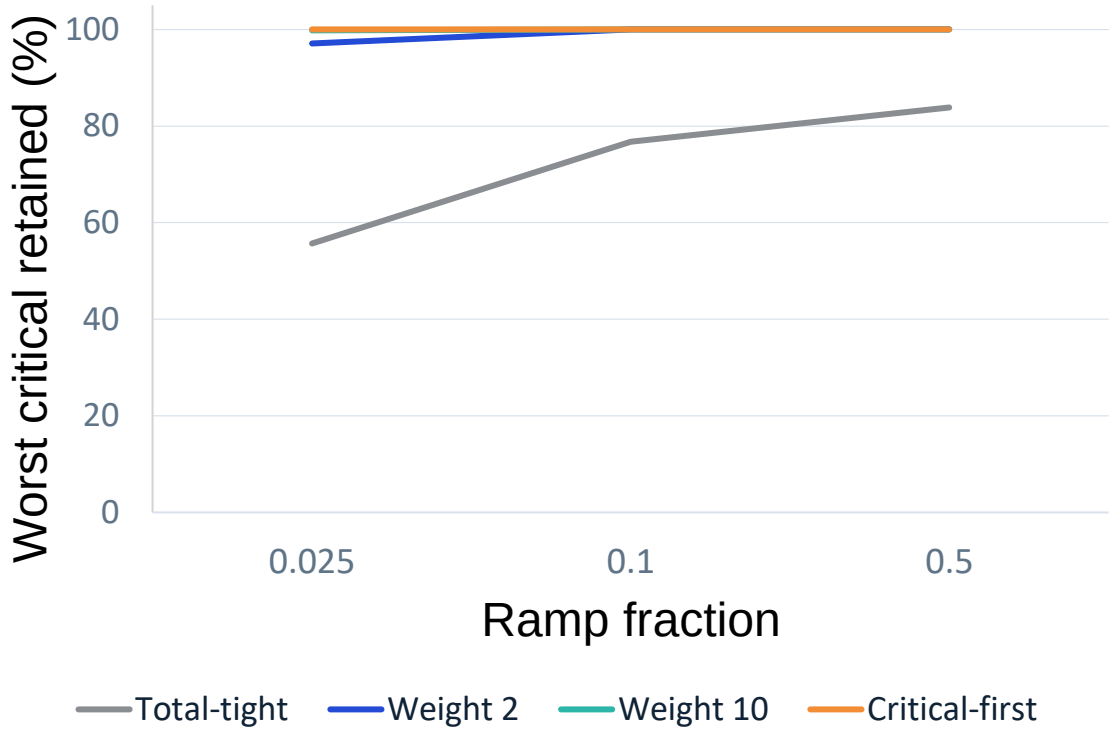
SENSITIVITY

Critical service across response speeds

Case 39



Case 118



Faster response changes the result. Group sizes also vary over 10%, 25% and 50% of load buses.

VERIFICATION

Correctness and boundaries of the results

16

Extension tests pass

26

State probes match untouched group code

Exact replay

Saved protection maxima agree with fresh exhaustive searches

Evaluation boundaries

Shared attack cohorts support fair comparisons.
Small protection cases use exhaustive search.

Original benchmark: case 162 at k = 8 is missing after an infeasible response.

The full paper benchmark remains incomplete.

Numerical checks support these measured studies. They do not remove the search and model limitations.

CODE

Where the extensions are implemented

grid_model.py

Physical response, optional service weights and critical-first floors

extensions.py

Attack tables, protection selection and timed traces

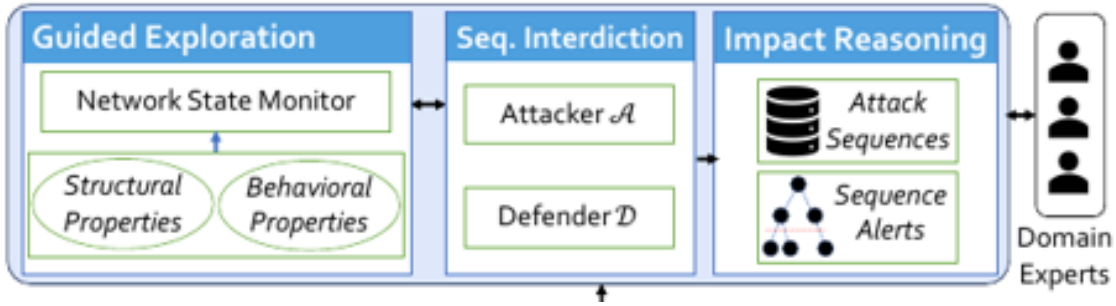
run_extensions.py

Scenarios, common cohorts, sweeps and summaries

audit_extensions.py

Fresh replay and matched-precision comparisons

Same physical defender



New experiments wrap the group implementation's existing response model.

Code and results: group-sequin/EXTENSIONS_README.md

DEMONSTRATION

Demonstration and interpretation

1. Protected lines and adaptive attacks

Case 14

Demand seed 100. Protected line IDs: 4, 3. Adaptive attack: 15, 1.

Round	Failed lines	Total shed MW
0		0.000
1	15	20.317
2	15, 1	395.317

Protection

Protected lines and the replacement worst attack

Timing

The same attack under even, early and late windows

Priority

One selected gain and one selected harm

Open output/demonstration.html. Selectors replay saved traces; CSV and JSON retain the detailed evidence.

CONCLUSIONS

Improvement, mixed evidence and failure

Protection

Useful gains

Case 14 improves; held-out and stress failures remain.

Timing

Different outcomes

Failure spacing changes final loss and the energy proxy.

Critical service

Benefits with costs

Selected service improves; other loss and later harm can increase.

The contribution is a measured investigation of when these choices help, tie or fail.

FUTURE WORK

Next steps motivated by the results

01 Stress failures

Train protection across demand and response-speed uncertainty.

02 Sampled larger cases

Search more strongly for attacks against critical demand. Vary priority groups.

03 Model assumptions

Compare restart rules and dispatch ties. Add lookahead, continuous ramps, restoration, storage and AC checks.

The strongest next experiments follow the failures and uncertainties already observed.

POWERFUL PRESENTATIONS!

THANK YOU

Neal Creative ©

Neal Creative | click & **Learn more**